

FileServer

Progettazione web service

Indice

1	Panoramica generale	2
2	Authentication Server	2
2.1	Architettura interna	2
2.1.1	Router	2
2.1.2	Controller	2
2.2	Database	2
2.3	Endpoint	3
2.3.1	POST /api/auth/register	3
2.3.2	POST /api/auth/login	3
2.3.3	POST /api/auth/verify	4
3	File Server	4
3.1	Architettura interna	4
3.1.1	Router	4
3.1.2	Controller	4
3.1.3	Middleware	4
3.1.4	Middleware di Autenticazione	4
3.1.5	Middleware di Storage	4
3.2	Database	5
3.3	Endpoint - Files	5
3.3.1	GET /api/files	5
3.3.2	POST /api/files/upload	6
3.3.3	GET /api/files/download/:id	6
3.3.4	PUT /api/files/:id/sposta	7
3.3.5	DELETE /api/files/:id	7
3.4	Endpoint — Cartelle	7
3.4.1	GET /api/cartelle	7
3.4.2	POST /api/cartelle	8
3.4.3	PUT /api/cartelle/:id/sposta	8
3.4.4	DELETE /api/cartelle/:id	8
4	Interazione tra i due server	9

1 Panoramica generale

Il progetto è composto da due microservizi REST indipendenti, sviluppati con **Node.js** ed **Express**, che comunicano tra loro tramite HTTP.

Servizio	Porta	Responsabilità
Authentication Server	3000	Gestione utenti, registrazione, login, emissione e verifica token JWT
File Server	4000	Upload, download, organizzazione file; serve il FrontEnd statico

Ogni server ha il proprio database MySQL separato. L'autenticazione tra i due avviene tramite **JSON Web Token (JWT)**: il File Server non verifica mai il token autonomamente, ma lo delega all'Authentication Server tramite una chiamata HTTP interna.

2 Authentication Server

2.1 Architettura interna

2.1.1 Router

Definisce tre route, tutte di tipo POST, collegate ai rispettivi metodi del controller. Nessuna route richiede autenticazione (sono route pubbliche).

2.1.2 Controller

Contiene la logica applicativa suddivisa in tre funzioni:

- **register** — valida i dati, effettua l'hash della password, controlla l'unicità dell'email e inserisce l'utente nel database.
- **login** — cerca l'utente per email, confronta la password con l'hash e genera un token JWT firmato con un segreto.
- **verify** — riceve un token, lo decodifica, e controlla che l'utente esista ancora nel database.

2.2 Database

Il database di autenticazione contiene una singola tabella:

```
CREATE TABLE utenti (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  nome        VARCHAR(100)  NOT NULL,  
  email       VARCHAR(100)  NOT NULL UNIQUE,  
  password    VARCHAR(255)  NOT NULL,    -- hash bcrypt  
  creato_il   TIMESTAMP     DEFAULT CURRENT_TIMESTAMP  
);
```

2.3 Endpoint

2.3.1 POST /api/auth/register

Registra un nuovo utente .

Corpo richiesta (JSON):

Campo	Tipo	Obbligatorio	Note
nome	string	✓	Nome visualizzato
email	string	✓	Email valida, non già registrata
password	string	✓	Minimo 6 caratteri

Risposta 200 OK:

```
{
  "messaggio": "Utente registrato con successo",
  "utente": {
    "id": 1,
    "nome": "Paolo",
    "email": "paolo@gmail.com"
  }
}
```

2.3.2 POST /api/auth/login

Effettua il login e restituisce un token JWT, che dopo 24h scade.

Corpo richiesta (JSON):

Campo	Tipo	Obbligatorio
email	string	✓
password	string	✓

Risposta 200 OK:

```
{
  "messaggio": "Login effettuato con successo",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IiI6...\"",
  "utente": {
    "id": 1,
    "nome": "Lorenzo",
    "email": "lorenzo@gmail.com"
  }
}
```

2.3.3 POST /api/auth/verify

Verifica la validità di un token JWT.

Corpo richiesta (JSON):

Campo	Tipo	Obbligatorio	Note
token	string	✓	Il token JWT da verificare

Risposta 200 OK:

```
{
  "valido": true,
  "utente": {
    "id": 1,
    "nome": "Giordano",
    "email": "giordano@gmail.com"
  }
}
```

3 File Server

3.1 Architettura interna

3.1.1 Router

Definisce le route, di vario tipo, per gestire file e cartelle. Viene eseguito il middleware di autenticazione in tutti i casi prima dei controller e nella route di upload dei file viene eseguito anche il middleware di storage.

3.1.2 Controller

Sono due, uno per i file e uno per le cartelle e contengono la logica applicativa per ottenere le info sui file, scaricarli, uploadarli, cancellarli e spostarli e creare cartelle, cancellarle, ottenere il percorso e spostarle.

3.1.3 Middleware

3.1.4 Middleware di Autenticazione

È il punto di collegamento tra i due server. Viene eseguito prima di ogni controller di route con autenticazione e permette di verificare il token JWT; in base all'esito della verifica aggiunge alla richiesta l'identificativo dell'utente oppure restituisce un errore.

3.1.5 Middleware di Storage

Configura Multer per la gestione degli upload definendo destinazione, file accettati e dimensione massima, e permette di effettuare l'upload effettivo.

3.2 Database

```
CREATE TABLE cartelle (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  nome        VARCHAR(100) NOT NULL,  
  utente_id   INT          NOT NULL,  
  percorso    VARCHAR(500) NOT NULL, -- es. /Documenti/Scuola  
  creata_il   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE KEY uq_cartella_percorso_utente (utente_id, percorso)  
);  
  
CREATE TABLE files (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  nome_originale VARCHAR(255) NOT NULL, -- nome scelto dall'  
    utente  
  nome_salvato  VARCHAR(255) NOT NULL, -- nome univoco su disco  
  dimensione   INT          NOT NULL, -- in byte  
  mimetype     VARCHAR(100),  
  utente_id    INT          NOT NULL,  
  cartella_id  INT          DEFAULT NULL, -- NULL = file in  
    root  
  caricato_il  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (cartella_id) REFERENCES cartelle(id)  
);
```

3.3 Endpoint - Files

Tutte le route richiedono l'header Authorization: Bearer <token>.

3.3.1 GET /api/files

Restituisce la lista dei file dell'utente. Senza parametri mostra i file in root; con `cartella_id` filtra per cartella specifica.

Parametri richiesta:

Parametro	Tipo	Note
<code>cartella_id</code>	integer	Opzionale. Se assente, elenca i file in root

Risposta 200 OK:

```
{  
  "files": [  
    {  
      "id": 12,  
      "nome_originale": "relazione.pdf",  
      "dimensione": 204800,  
      "mimetype": "application/pdf",  
      "cartella_id": 5,  
      "caricato_il": "2026-04-28T14:35:22.000Z"
```

```
}
]
}
```

3.3.2 POST /api/files/upload

Carica un file sul server. La richiesta deve essere inviata come `multipart/form-data` perché presenta un file e un intero.

Campi form:

Campo	Tipo	Obbligatorio	Note
file	file	✓	Il file da caricare
cartella_id	integer		Se assente, il file va in root

Risposta 200 OK:

```
{
  "messaggio": "File caricato con successo",
  "file": {
    "id": 12,
    "nome_originale": "prova",
    "dimensione": 204800,
    "mimetype": "application/pdf",
    "cartella_id": 5,
    "caricato_il": "2026-04-28T14:35:22.000Z"
  }
}
```

3.3.3 GET /api/files/download/:id

Scarica un file. Risponde con il contenuto binario e imposta il nome originale.

Parametri percorso:

Parametro	Tipo	Note
id	integer	ID del file da scaricare

Risposta:

Status	Descrizione
200 OK	File scaricato con successo

3.3.4 PUT /api/files/:id/sposta

Sposta un file in una cartella diversa o in root.

Corpo richiesta (JSON):

Campo	Tipo	Note
cartella_id	integer null	Nuova cartella; null per spostare in root

Risposta 200 OK:

```
{
  "messaggio": "Spostamento riuscito"
}
```

3.3.5 DELETE /api/files/:id

Elimina un file sia dal database che dal disco.

Risposta 200 OK:

```
{
  "messaggio": "File eliminato con successo"
}
```

3.4 Endpoint — Cartelle

Tutte le route richiedono l'header **Authorization: Bearer <token>**.

3.4.1 GET /api/cartelle

Restituisce tutte le cartelle dell'utente, ordinate per percorso.

Risposta 200 OK:

```
{
  "cartelle": [
    { "id": 1, "nome": "Documenti",
      "percorso": "/Documenti",
      "creata_il": "2026-04-28T14:35:22.000Z" },
    { "id": 2, "nome": "Scuola",
      "percorso": "/Documenti/Scuola",
      "creata_il": "2026-04-28T14:35:22.000Z" }
  ]
}
```

3.4.2 POST /api/cartelle

Crea una nuova cartella. Il percorso completo viene calcolato automaticamente concatenando `percorso_padre` e `nome`.

Request body (JSON):

Campo	Tipo	Obbligatorio	Note
<code>nome</code>	string	✓	Nome della cartella
<code>percorso_padre</code>	string		Percorso padre. Se assente o /, crea in root

Risposta 200 OK:

```
{
  "messaggio": "Cartella creata con successo",
  "cartella": {
    "id": 1,
    "nome": "Documenti",
    "percorso": "/Documenti",
    "creata_il": "2026-04-28T14:35:22.000Z"
  }
}
```

3.4.3 PUT /api/cartelle/:id/sposta

Sposta una cartella e tutte le sue sottocartelle in una nuova posizione. Non è possibile spostare una cartella dentro sé stessa o in una propria sottocartella.

Corpo richiesta (JSON):

Campo	Tipo	Note
<code>percorso_destinazione</code>	string	null per spostare in root

Risposta 200 OK:

```
{
  "messaggio": "Spostamento riuscito"
}
```

3.4.4 DELETE /api/cartelle/:id

Elimina una cartella solo se non contiene niente.

Risposta 200 OK:

```
{
  "messaggio": "Cartella eliminata"
}
```

4 Interazione tra i due server

Il diagramma seguente mostra il flusso completo di una richiesta autenticata:

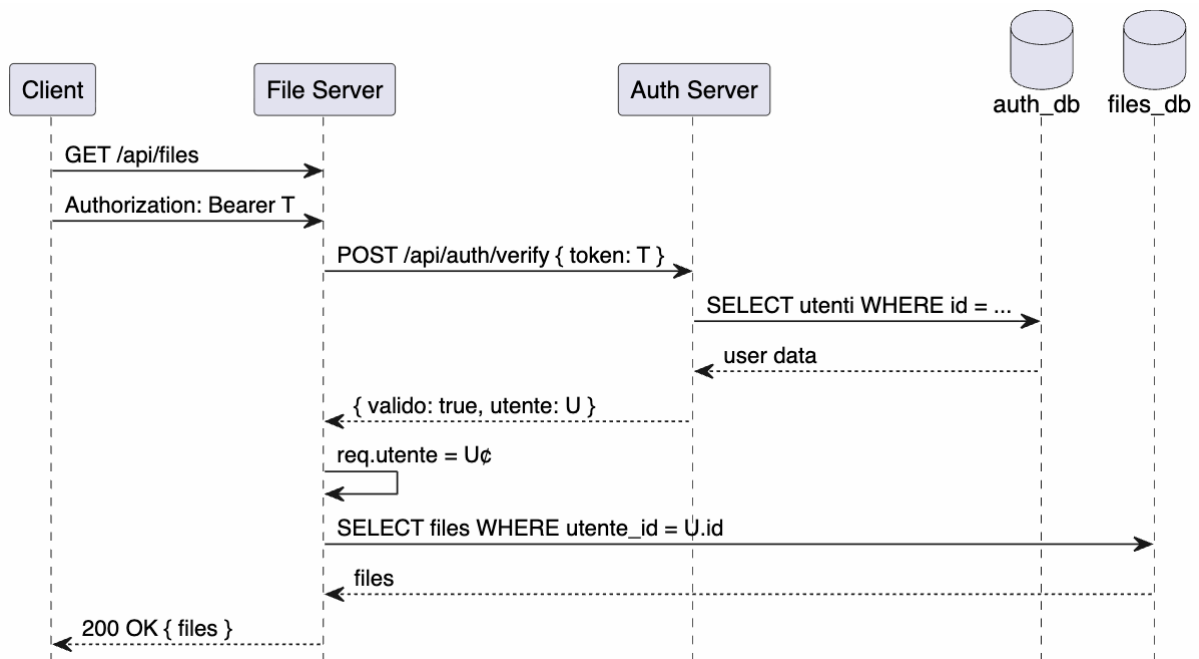


Figura 1: Flusso di autenticazione inter-server